BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION
Robotics and AI

# MPC-MAP Assignment No. 3 - Report

**Author:** Tomáš Linek

**Date:** 10.04.2026

## Task 1

The prediction function has been implemented. The predicted pose of the particles is based on the calculated linear and angular robot speed from the control input. To increase the variance of the particles, I added noise to the new predicted position of each particle. I also tried adding noise to the calculated robot speeds, but I got worse localization results.

## Task 2

The functions compute_lidar_measurement and weight_particles have been implemented. For weight calculation, two methods have been implemented: the first using a normal distribution and the second using Euclidean distance. Robot localization performed better using Euclidean distance, so I chose this method.

## Task 3

The function resample_particles has been implemented using Thrun's heuristic algorithm. To mitigate the kidnapped robot problem, 10% of all particles are set to a random position on the map. For random distribution of particles across the map, a new function input was added to access the map limits.

## Task 4

The most important parameters are the number of randomly distributed particles during the resampling step and the noise values in the prediction step. A low number of randomly distributed particles leads to the estimated pose getting stuck in a wrong position and failing to converge to the robot's actual position. On the other hand, too high a number leads to poor pose estimation. With too little noise in the prediction step, the particles do not spread enough, and the filter gets stuck in similar areas of the map. However, with too much noise, the localization becomes unstable and the estimated pose is less accurate compared to the true pose. The major issue was to find the cause of unexpected failures in the position estimate. These failures were caused by the weights being set to NaN in cases where the LiDAR channel did not detect any wall.

Figures 1, 2, and 3 show the testing of the implemented particle filter, where the robot follows the path based on the estimated pose from the particle filter.

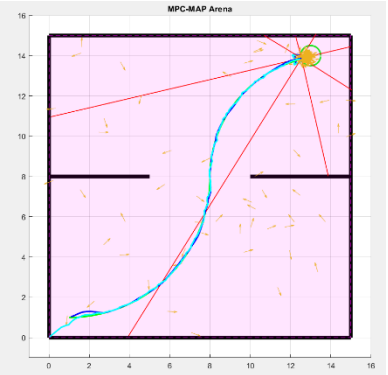


*Figure 1 – First path following*

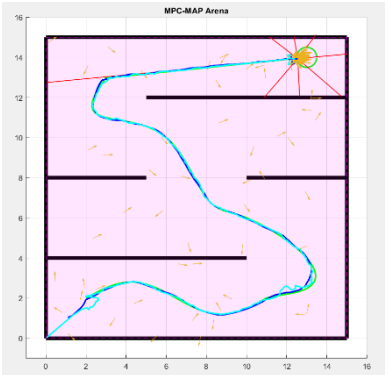


*Figure 2 – Second path following*

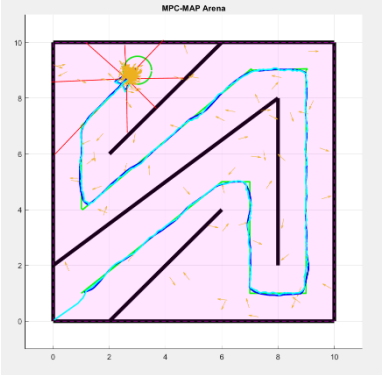


*Figure 3 – Third path following*